LAPORAN PRAKTIKUM PEMOGRAMAN BERORIENTASI OBJEK

"Abstraction"

Disusun Oleh:

Sofian Arba'i (2511533029)

Dosen Pengampu:

Afdhal Dinilhak, M.Kom.

Asisten Praktikum:

M Dawi Syauqi



DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

Praktikum Pemrograman Berorientasi Objek Pekan 6

”Abstraction”

1. Tahapan Praktikum

1.1 Persiapan Praktikum

Beberapa hal yang harus dipersiapkan sebelum mengerjakan semua langkah dan latihan dari praktikum pekan 6 Abstraction sebagai berikut:

a. Membuat Proyek Baru

Hal pertama yang harus dipersiapkan untuk praktium adalah membuat sebuah proyek java baru sendiri di VS Code tanpa build tool. Nama proyek java tersebut yaitu Abtraction_2511533029

b. Membuat Folder Package

Selanjutnya yaitu membuat beberapa folder package sebagai tempat file java nnti akan dikerjakan dan menjadi lebih terstruktur. Membuat folder package dengan klik kanan src lalu New Java Package, lalu ketik nama berikut satu per satu.

- shop/model
- shop/shipping
- shop/discount
- shop/payment
- shop/service
- shop/app

c. Salin Alat Bantu

Alat bantu pada praktikum pekan 6 terkumpul dalam satu file zip. Ekstrak isi dari file zip dan salin ke dalam folder proyek java sesuai dengan

struktur yang telah ditentukan. Lalu menjalankan SelfCheck dan akan mengeluarkan output hasil pemeriksaan lulus 0/23.

1.2 Langkah Kerja Praktikum (Latihan)

Pada praktikum pekan 6 membahasa beberapa materi yaitu abctract class, method, interface dan lainnya. Beberapa materi tersebut terbungkus dalam satu proyek java dan berbentuk latihan yang dimulai dari L1-L5. Berikut adalah langkah kerja praktikum (Latihan) pekan 6.

1.2.1 Latihan (L1)

Pada latihan 1 membahas abstract class dan abstract method dengan membuat beberapa class baru yang berhubungan dengan proyek java.

a. Class Product

```java
package shop.model;

public abstract class Product {
    private final String sku;
    private final String name;
    private final long price; // in ruptah

    protected Product(String sku, String name, long
price) {
        if (sku == null || sku.isBlank()) {
            throw new IllegalArgumentException("SKU
must not be blank");
        }
        if (name == null || name.isBlank()) {
            throw new IllegalArgumentException("Name
must not be blank");
        }
        if (price < 0) {
            throw new IllegalArgumentException("Price
must not be negative");
        }
        this.sku = sku;
        this.name = name;
        this.price = price;
    }

    public String getSku() {
        return sku;
    }

    public String getName() {
        return name;
    }

    public long getPrice() {
        return price;
    }

    /** Price the customer actually pays. Subclasses
may override it. */
    public long getFinalPrice() {
        return price;
    }

    /** Every concrete product must state its
category. */
    public abstract String category();

    /** A concrete method that relies on the abstract
method category(). */
    public String getInfo() {
        return "[" + category() + "] " + name + " -
Rp" + getFinalPrice();
    }
}
```

Kode Program 2.1

Class Product adalah *abstract class* sehingga tidak dapat diinstansiasi langsung. Pemanggilan new Product(...) ditolak compiler, dan pesan error itu yang dilampirkan di laporan L1. Class ini menjadi kerangka bersama bagi PhysicalProduct dan DigitalProduct, yang berbagi field dan konstruktor, yaitu sku, name, dan price. Ketiga field bersifat private final sehingga hanya diisi sekali lewat konstruktor dan diakses lewat getter (enkapsulasi). Harga bertipe long

karena rupiah berupa bilangan bulat. Konstruktornya protected agar hanya dipanggil subclass lewat super(...), dan memuat validasi yang melempar IllegalArgumentException jika SKU atau nama kosong atau harga negatif.

Method getFinalPrice() bersifat konkret dan tidak di-final, sehingga subclass boleh meng-override-nya, seperti DigitalProduct yang menerapkan diskon 10% pada L3. Method category() bersifat *abstract*, jadi setiap subclass konkret wajib mengisinya sendiri, misalnya "Fisik" atau "Digital". Method getInfo() konkret tetapi memanggil category() dan getFinalPrice(), sehingga cukup ditulis sekali di Product namun hasilnya menyesuaikan jenis objek aslinya. Inilah dasar polimorfisme sekaligus cikal bakal *Template Method pattern* pada T2.

b. Class PhysicalProduct



```
package shop.model;

import shop.shipping.Shippable;

public class PhysicalProduct extends Product
implements Shippable {
    private final int weightGrams;

    public PhysicalProduct(String sku, String name,
long price, int weightGrams) {
        super(sku, name, price);
        if (weightGrams <= 0) {
            throw new IllegalArgumentException("Weight
must be greater than zero");
        }
        this.weightGrams = weightGrams;
    }

    @Override
    public int getWeightGrams() {
        return weightGrams;
    }

    @Override
    public String category() {
        return "Fisik";
    }
}
```

Kode Program 2.2

Class PhysicalProduct adalah subclass konkret dari Product (extends), sehingga mewarisi sku, name, price, getFinalPrice(), dan getInfo(). Konstruktornya memanggil super(sku, name, price) lebih dulu untuk memakai validasi di Product, lalu menambah validasi sendiri: weightGrams harus lebih dari nol, jika tidak dilempar IllegalArgumentException. Field weightGrams bersifat private final dan diakses lewat getter. Method abstract category() dari Product diisi dengan "Fisik" lewat @Override, sehingga class ini tidak lagi abstract dan bisa diinstansiasi.

Class ini juga implements Shippable, yaitu interface yang menjadi kontrak bagi barang yang bisa dikirim. Karena itu getWeightGrams() wajib ada dan diberi @Override. Di sinilah terlihat perbedaan abstract class dan interface: extends Product menjawab "produk ini jenis apa" dan mewariskan field serta perilaku, sedangkan implements Shippable menjawab "produk ini bisa apa", yaitu memiliki berat untuk dihitung ongkirnya. Satu class hanya boleh mewarisi satu class, tetapi boleh mengimplementasikan banyak interface. DigitalProduct sengaja tidak mengimplementasikan Shippable karena e-book tidak dikirim, dan hal ini membuat CheckoutService bisa menghitung berat hanya dari barang fisik tanpa mengetahui class konkretnya (*program to an interface*).

c. Class DigitalProduct

```
package shop.model;

import shop.discount.Discountable;

public class DigitalProduct extends Product implements
Discountable {
    private static final int DIGITAL_DISCOUNT_PERCENT
= 10;
    private final String downloadUrl;

    public DigitalProduct(String sku, String name,
long price, String downloadUrl) {
        super(sku, name, price);
        if (downloadUrl == null ||
downloadUrl.isBlank()) {
            throw new
IllegalArgumentException("DownloadUrl tidak boleh
kosong");
        }
        this.downloadUrl = downloadUrl;
    }

    public String getDownloadUrl() {
        return downloadUrl;
    }

    @Override
    public String category() {
        return "Digital";
    }

    @Override
    public int discountPercent() {
        return DIGITAL_DISCOUNT_PERCENT;
    }

    @Override
    public long getFinalPrice() {
        return applyDiscount(getPrice());
    }

}
```

Kode Program 2.3

Class DigitalProduct adalah subclass konkret dari Product (extends) yang mewarisi sku, name, dan price. Konstruktornya memanggil super(sku, name, price) lebih dulu, lalu menambah validasi sendiri: downloadUrl tidak boleh null atau kosong, jika tidak dilempar IllegalArgumentException. Field downloadUrl bersifat private final dan diakses lewat getDownloadUrl(). Method abstract category() diisi dengan "Digital" lewat @Override, sehingga class ini tidak lagi abstract. Berbeda dengan PhysicalProduct, class ini tidak

mengimplementasikan Shippable karena e-book tidak dikirim dan tidak punya berat.

Class ini juga implements Discountable, interface yang memberi kontrak diskon pada L3. Kontraknya hanya satu method abstract, discountPercent(), yang di sini mengembalikan konstanta DIGITAL_DISCOUNT_PERCENT (10). Konstanta itu private static final karena nilainya tetap dan milik class, bukan milik tiap objek. Method getFinalPrice() milik Product di-override agar memanggil applyDiscount(getPrice()). Method applyDiscount() adalah *default method* di Discountable, jadi DigitalProduct memakainya tanpa menulis ulang rumusnya. Hasilnya harga e-book 50000 menjadi 45000. CheckoutService tidak perlu diubah, karena subtotal() sudah memanggil getFinalPrice() dan polimorfisme memilih versi yang tepat sesuai jenis objeknya.

d. Class ShippingService



```
package shop.shipping;

public abstract class ShippingService {
    private final String name;

    protected ShippingService(String name) {
        if (name == null || name.isBlank()) {
            throw new IllegalArgumentException("Name
must not be blank");
        }
        this.name = name;
    }

    public String getName() {
        return name;
    }

    /** Rate in rupiah per started kilogram. */
    protected abstract long ratePerKg();

    /** Weight is rounded up: 700 g -> 1 kg, 1001 g ->
2 kg. */
    public final long calculateCost(int weightGrams) {
        if (weightGrams <= 0) {
            return 0;
        }
        long kilograms = (weightGrams + 999L) / 1000;
        return kilograms * ratePerKg();
    }

}
```

Kode Program 2.4

Class ShippingService adalah *abstract class* yang menjadi kerangka bersama bagi semua kurir, seperti RegularShipping dan ExpressShipping. Field name bersifat private final dan diisi lewat konstruktor protected, sehingga hanya dapat dipanggil subclass melalui super(...). Konstruktor itu menolak nama null atau kosong dengan IllegalArgumentException, dan nama diakses lewat getName(). Method ratePerKg() bersifat *abstract* karena tarif tiap kurir berbeda, jadi setiap subclass wajib mengisinya

sendiri, misalnya tarif Reguler dan Ekspres. Method ini sengaja tidak di-final, sebab tarif memang harus bisa berbeda.

Method calculateCost(int weightGrams) bersifat konkret sekaligus final. Rumus ongkir ditulis sekali di sini dan dikunci agar subclass tidak dapat menggantinya, misalnya membuat kurir yang selalu mengembalikan nol. Mencoba meng-override method ini menghasilkan error compiler, dan itulah yang dilampirkan di laporan pada L4. Rumusnya mengembalikan 0 untuk berat nol atau negatif, selain itu membulatkan berat ke atas per kilogram dengan (weightGrams + 999L) / 1000, lalu mengalikannya dengan ratePerKg(). Method final ini memanggil method abstract ratePerKg() yang diisi subclass, sehingga rumus tetap sama tetapi tarifnya menyesuaikan kurir. Pola ini adalah *Template Method* yang akan muncul lagi pada T2 lewat send(...) yang bersifat final.

e. Class RegularShipping dan ExpressShipping



```
package shop.shipping;

public class RegularShipping extends ShippingService {
    public RegularShipping() {
        super("Regular");
    }

    @Override
    protected long ratePerKg() {
        return 10_000;
    }

}
```

Kode Program 2.5

Class RegularShipping adalah subclass konkret dari ShippingService (extends) yang mewakili kurir Reguler. Konstruktornya tanpa parameter dan memanggil super("Regular"), sehingga nama kurir terisi dan melewati validasi di class induk. Method abstract ratePerKg() diisi lewat @Override dengan tarif 10.000 rupiah per kilogram, dan hanya tarif itu yang ditulis di sini karena rumus ongkir sudah dikunci final pada calculateCost() milik induknya. Misalnya, berat 700 gram dibulatkan menjadi 1 kg sehingga ongkirnya 10.000, dan 1001 gram menjadi 2 kg sehingga ongkirnya 20.000. Karena CheckoutService menerima tipe ShippingService,

polimorfisme memilih tarif yang sesuai tanpa perlu mengetahui kurir konkretnya (*program to an interface*).

```java
package shop.shipping;

public class ExpressShipping extends ShippingService {
    public ExpressShipping() {
        super("Express");
    }

    @Override
    protected long ratePerKg() {
        return 18_000;
    }

}
```

Kode Program 2.6

Class ExpressShipping adalah subclass konkret dari ShippingService (extends) yang mewakili kurir Ekspres. Konstruktornya tanpa parameter dan memanggil super("Express"), sehingga nama kurir terisi dan melewati validasi di class induk. Method abstract ratePerKg() diisi lewat @Override dengan tarif 18.000 rupiah per kilogram, lebih mahal daripada RegularShipping yang 10.000. Hanya tarif itu yang berbeda, karena rumus ongkir sudah dikunci final pada calculateCost() milik induknya. Misalnya, berat 700gram dibulatkan menjadi 1 kg sehingga ongkirnya 18.000, dan 1001gram menjadi 2 kg sehingga ongkirnya 36.000. Karena RegularShipping dan ExpressShipping sama-sama bertipe ShippingService, CheckoutService dapat menerima keduanya dan polimorfisme memilih tarif yang sesuai (*program to an interface*).

1.2.2 Latihan (L2)

Pada latihan 2 membahas Interface dan program to an interface dengan membuat beberapa class baru yang berhubungan dengan proyek java sebagai berikut.

a. Interface Shippable

```java
package shop.shipping;

/** Capability: anything that has weight and can be
shipped. */
public interface Shippable {
    int getWeightGrams();

}
```

Kode Program 2.7

Interface Shippable adalah kontrak untuk barang yang bisa

dikirim, berisi satu method abstract getWeightGrams() tanpa isi (otomatis public abstract). Beberapa class lain bisa mengimplementasikan class shippable ini.

b. Interface PaymentMethod

```
package shop.payment;

/** Contract for every payment method. */
public interface PaymentMethod {
    String getName();

    /** @return true if the payment succeeds */
    boolean pay(long amount);

}
```

Kode Program 2.8

Interface PaymentMethod adalah kontrak bagi semua metode pembayaran, berisi method abstract getName() dan pay(long amount) yang wajib ditulis sendiri oleh tiap class yang mengimplementasikannya, seperti EWalletPayment dan BankTransferPayment.

c. Class EWalletPayment

```
package shop.payment;

public class EWalletPayment implements
RefundablePayment {
    private final String provider;
    private long balance;

    public EWalletPayment(String provider, long
balance) {
        if (provider == null || provider.isBlank()) {
            throw new
IllegalArgumentException("provider tidak boleh
kosong");
        }
        if (balance < 0) {
            throw new IllegalArgumentException("saldo
tidak boleh negatif");
        }
        this.provider = provider;
        this.balance = balance;
    }

    @Override
    public String getName() {
        return "E-Wallet " + provider;
    }

    public long getBalance() {
        return balance;
    }

    @Override
    public boolean pay(long amount) {
        if (amount <= 0 || amount > balance) {
            return false;
        }
        balance -= amount;
        return true;
    }

    @Override
    public boolean refund(long amount) {
        if (amount <= 0) {
            return false;
        }
        balance += amount;
        return true;
    }
}
```

Kode Program 2.9

Class EWalletPayment implementasi RefundablePayment (turunan PaymentMethod), sehingga wajib menulis getName(), pay(), dan refund(). Konstruktornya menolak provider kosong dan saldo negatif. Field provider bersifat private final karena tetap, sedangkan balance tidak final karena berubah tiap transaksi. Method pay() gagal jika jumlah tidak positif atau melebihi saldo, selain itu saldo dikurangi. Method refund() gagal jika jumlah tidak

positif, selain itu saldo bertambah. Karena bertipe interface, objek ini dapat diterima CheckoutService tanpa class konkretnya diketahui (*program to an interface*).

d. Class BankTransferPayment

```java
package shop.payment;

public class BankTransferPayment implements
PaymentMethod {
    private final String bankName;

    public BankTransferPayment(String bankName) {
        if (bankName == null || bankName.isBlank()) {
            throw new IllegalArgumentException("Nama
bank tidak boleh kosong");
        }
        this.bankName = bankName;
    }

    @Override
    public String getName() {
        return "Transfer " + bankName;
    }

    @Override
    public boolean pay(long amount) {
        return amount > 0;
    }

}
```

Kode Program 2.10

Class BankTransferPayment mimplementasi PaymentMethod, sehingga wajib menulis getName() dan pay(). Konstruktornya menolak nama bank kosong, dan field bankName bersifat private final karena tidak berubah. Method pay() hanya mengembalikan true jika jumlah positif karena ini simulasi tanpa saldo. Class ini sengaja tidak mengimplementasikan RefundablePayment, sehingga pemanggilan checkout.refund(...) dengan objek ini ditolak compiler, dan itulah error yang dilampirkan pada L3.

e. Class CheckoutService

```java
package shop.service;

import java.util.List;
import shop.model.Product;
import shop.payment.PaymentMethod;
import shop.payment.RefundablePayment;
import shop.shipping.Shippable;
import shop.shipping.ShippingService;

public class CheckoutService {
    public long subtotal(List<Product> items) {
        long total = 0;
        for (Product product : items) {
            total += product.getFinalPrice();
        }
        return total;
    }

    public int totalWeight(List<Product> items) {
        int grams = 0;
        for (Product product : items) {
            if (product instanceof Shippable
shippable) {
                // a capability, not a class
                grams += shippable.getWeightGrams();
            }
        }
        return grams;
    }
```

Kode Program 2.11

```java
public long total(List<Product> items, ShippingService
shipping) {
        long itemsSubtotal = subtotal(items);
        int weight = totalWeight(items);
        long shippingCost =
shipping.calculateCost(weight);

        return itemsSubtotal + shippingCost;
    }

    public boolean checkout(List<Product> items,
ShippingService shipping, PaymentMethod payment) {
        if (items == null || items.isEmpty()) {
            return false;
        }

        long totalAmount = total(items, shipping);
        return payment.pay(totalAmount);
    }

    public boolean refund(RefundablePayment payment,
long amount) {
        return payment.refund(amount);
    }
}
```

Class CheckoutService mengatur proses belanja dan sengaja hanya bergantung pada tipe abstrak, yaitu Product, Shippable, ShippingService, PaymentMethod, dan RefundablePayment (*program to an interface*). Method subtotal() menjumlahkan getFinalPrice() semua produk. Method totalWeight() memakai instanceof Shippable untuk menjumlahkan berat hanya dari barang yang punya kemampuan dikirim. Method total() menjumlahkan subtotal dengan ongkir dari shipping.calculateCost(weight). Method checkout() mengembalikan false jika keranjang kosong. Method refund() menerima tipe RefundablePayment, bukan PaymentMethod, sehingga pembayaran yang tidak mendukung refund, seperti BankTransferPayment, ditolak compiler sebelum program berjalan.

f. ShopApp (Mulai ditulis)

```java
package shop.app;

import java.util.List;
import shop.model.DigitalProduct;
import shop.model.PhysicalProduct;
import shop.model.Product;
import shop.service.CheckoutService;
import shop.shipping.ExpressShipping;
import shop.shipping.RegularShipping;
import shop.shipping.ShippingService;

public class ShopApp {
    public static void main(String[] args) {
        // L1: abstract class and abstract method
        Product coffee = new PhysicalProduct("KOPI-
001", "Kopi Arabika Gayo 250g", 85_000, 300);
        Product rendang = new PhysicalProduct("RDG-
002", "Rendang Kemasan 250g", 120_000, 400);
        Product ebook = new DigitalProduct("EBK-003",
"E-book Resep Kue Nusantara", 50_000,
                "https://toko-umkm.example/unduh/ebk-
003");

        List<Product> cart = List.of(coffee, rendang,
ebook);
        System.out.println("== Katalog ==");
        for (Product product : cart) {
            System.out.println(product.getInfo());
        }

        // L2: interface and program to an interface
        CheckoutService checkout = new
CheckoutService();
        ShippingService regular = new
RegularShipping();
        ShippingService express = new
ExpressShipping();
        System.out.println("\n== Ringkasan ==");
        System.out.println("Subtotal      : Rp" +
checkout.subtotal(cart));
        System.out.println("Berat kiriman : " +
checkout.totalWeight(cart) + " g");
        System.out.println("Total Reguler : Rp" +
checkout.total(cart, regular));
        System.out.println("Total Ekspres : Rp" +
checkout.total(cart, express));
```

Kode Program 2.12

Bagian ShopApp ini berisi demo L1 dan L2 di dalam main. Pertama dibuat dua PhysicalProduct (kopi dan rendang) dan satu DigitalProduct (e-book), semuanya disimpan dalam List<Product>. Variabel bertipe Product bisa menampung ketiganya karena polimorfisme. Saat getInfo() dipanggil untuk katalog, tiap objek mencetak kategorinya sendiri ([Fisik] atau [Digital]) lewat method abstract category().

Setelah itu CheckoutService dan dua kurir dibuat dengan tipe ShippingService, sehingga RegularShipping dan ExpressShipping dapat dipakai bergantian (*program to an interface*). Ringkasannya menampilkan subtotal, total berat (700 gram, e-book tidak dihitung karena bukan Shippable), serta total dengan ongkir Reguler dan Ekspres. Berat 700 gram dibulatkan ke 1 kg, jadi ongkirnya 10.000 dan 18.000. Setelah diskon e-book (L3), angkanya menjadi subtotal 250000, total Reguler 260000, dan total Ekspres 268000.

1.2.3 Latihan (L3)

Pada latihan 3 membahas Interface lanjutan: default, static, private method dan interface inheritance dengan membuat beberapa class baru yang berhubungan dengan proyek java sebagai berikut.

a. Class Discountable



```
package shop.discount;

public interface Discountable {
    int discountPercent(); // abstract: each class
decides

    default long applyDiscount(long price) {
        return price - percentOf(price,
safePercent());
    }

    static long percentOf(long amount, int percent)
{
        return amount * percent / 100;
    }

    private int safePercent() {
        int percent = discountPercent();
        if (percent < 0) {
            return 0;
        }
        return Math.min(percent, 100);
    }

}
```

Kode Program 2.13

Interface Discountable menunjukkan empat jenis method dalam satu interface. discountPercent() adalah method abstract, sehingga tiap class menentukan persentasenya sendiri (misalnya DigitalProduct 10%). applyDiscount() adalah *default method* yang sudah berisi rumus harga dikurangi diskon, jadi class pengimplementasi memakainya tanpa menulis ulang dan tanpa dipaksa meng-override. percentOf() adalah *static method* yang dipanggil lewat nama interface, bukan lewat objek, karena hanya perhitungan pembantu tanpa data objek. safePercent() adalah *private method* yang hanya bisa dipakai di dalam interface ini. Fungsinya membatasi persentase ke

rentang 0 sampai 100 supaya diskon negatif atau di atas 100% tidak menghasilkan harga yang aneh.

b. Class DigitalProduct ikut promo



```
package shop.model;

import shop.discount.Discountable;

public class DigitalProduct extends Product
implements Discountable {
  .........
    @Override
    public int discountPercent() {
        return DIGITAL_DISCOUNT_PERCENT;
    }

    @Override
    public long getFinalPrice() {
        return applyDiscount(getPrice());
    }

}
```

Kode Program 2.14

Potongan ini adalah perubahan L3.b pada DigitalProduct, yaitu penambahan implements Discountable di samping extends Product. Class ini mewarisi dari satu class dan sekaligus memenuhi kontrak interface. Method discountPercent() wajib diisi karena abstract, dan di sini mengembalikan konstanta DIGITAL_DISCOUNT_PERCENT (10). Method getFinalPrice() milik Product di-override agar memanggil applyDiscount(getPrice()), yaitu default method dari Discountable, sehingga rumus diskon tidak ditulis ulang. Akibatnya harga e-book 50000 menjadi 45000, dan CheckoutService tidak perlu diubah karena subtotal() sudah memanggil getFinalPrice() lewat polimorfisme.

c. Interface (RefundablePayment) dan Interface (InstallmentPayment)



```
package shop.payment;

public interface RefundablePayment extends
PaymentMethod {
    boolean refund(long amount);
}
```

Kode Program 2.15

Interface RefundablePayment adalah interface turunan dari PaymentMethod (extends, bukan implements, karena interface mewarisi interface). Isinya satu method abstract tambahan, refund(long amount), sehingga class yang mengimplementasikannya wajib menulis getName() dan pay() dari induknya serta refund() miliknya sendiri, seperti EWalletPayment dan CreditCardPayment.

```java
package shop.payment;

public interface InstallmentPayment extends
PaymentMethod {
    int maxInstallmentMonths();

    /** Monthly amount, rounded up. */
    default long monthlyInstallment(long amount, int
months) {
        if (months < 1 || months >
maxInstallmentMonths()) {
            throw new
IllegalArgumentException("Months must be between 1
and " + maxInstallmentMonths());
        }
        return (amount + months - 1) / months;
    }

}
```

Kode Program 2.16

Interface InstallmentPayment adalah interface turunan dari PaymentMethod (extends) untuk pembayaran yang bisa dicicil. Method abstract maxInstallmentMonths() menentukan batas tenor yang diisi tiap class (kartu kredit 12 bulan). Method monthlyInstallment() adalah *default method* yang sudah berisi rumus cicilan, sehingga class pengimplementasi memakainya tanpa menulis ulang. Method ini melempar IllegalArgumentException jika tenor kurang dari 1 atau melebihi batas, lalu menghitung cicilan dengan (amount + months - 1) / months, yaitu pembulatan ke atas dengan aritmetika bilangan bulat. Contohnya 268000 dibagi 3 bulan menjadi 89334, bukan 89333. Kemampuan ini dipisah dari RefundablePayment karena tidak semua pembayaran bisa dicicil, dan CreditCardPayment dapat mengimplementasikan keduanya sekaligus (satu class boleh banyak interface).

d. Class CreditCardPayment

```java
package shop.payment;

public class CreditCardPayment implements
RefundablePayment, InstallmentPayment {
    private final String cardHolder;
    private final long creditLimit;
    private long usedCredit;

    public CreditCardPayment(String cardHolder, long
creditLimit) {
        if (cardHolder == null ||
cardHolder.isBlank()) {
            throw new IllegalArgumentException("Nama
Pemegang Kartu Tidak Boleh Kosong");
        }
        if (creditLimit < 0) {
            throw new
IllegalArgumentException("Credit Limit Tidak Boleh
Negatif");
        }
        this.cardHolder = cardHolder;
        this.creditLimit = creditLimit;
        this.usedCredit = 0;
    }

    @Override
    public String getName() {
        return "Kartu Kredit (" + cardHolder + ")";
    }
```

Kode Program 2.17

```java
    public long getUsedCredit() {
        return usedCredit;
    }

    @Override
    public boolean pay(long amount) {
        if (amount <= 0 || usedCredit + amount >
creditLimit) {
            return false;
        }
        usedCredit += amount;
        return true;
    }

    @Override
    public boolean refund(long amount) {
        if (amount <= 0 || amount > usedCredit) {
            return false;
        }
        usedCredit -= amount;
        return true;
    }

    @Override
    public int maxInstallmentMonths() {
        return 12;
    }

}
```

Class CreditCardPayment mengimplementasikan dua interface sekaligus, RefundablePayment dan InstallmentPayment, sehingga kartu kredit dapat di-refund dan dicicil. Class ini wajib menulis getName() dan pay() dari PaymentMethod (yang diwarisi kedua interface), refund() dari RefundablePayment, dan maxInstallmentMonths() dari InstallmentPayment, yang mengembalikan tenor maksimal 12 bulan. Method monthlyInstallment() tidak perlu ditulis karena sudah disediakan sebagai default method.

Konstruktornya menolak nama pemegang kartu kosong dan limit negatif. Field cardHolder dan creditLimit bersifat private final, sedangkan usedCredit berubah tiap transaksi. Method pay() gagal jika jumlah tidak positif atau pemakaian melebihi limit. Method refund() gagal jika jumlah tidak positif atau melebihi usedCredit, selain itu usedCredit berkurang. Berbeda dengan e-wallet yang memakai saldo, kartu kredit memakai limit dan mencatat pemakaiannya.

e. Lengkapi ShopApp dengan pembayaran

```
        EWalletPayment wallet = new
EWalletPayment("DompetKu ", 300_000);
        PaymentMethod transfer = new
BankTransferPayment("Bank Nagari");
        System.out.println("\n== Pembayaran ==");
        System.out.println(wallet.getName() +
"berhasil? " + checkout.checkout(cart, regular,
wallet));
        System.out.println("Sisa saldo: Rp" +
wallet.getBalance());
        System.out.println(transfer.getName() + "
berhasil? " + checkout.checkout(cart, express,
transfer));

        // L3: interface inheritance, multiple
interface, default method
        CreditCardPayment card = new
CreditCardPayment("Siti Rahma", 1_000_000);
        System.out.println(card.getName() + "
berhasil? " + checkout.checkout(cart, express,
card));
        System.out.println("Cicilan 3 bulan: Rp" +
card.monthlyInstallment(checkout.total(cart,
express), 3));

        System.out.println("\n== Refund e-book ==");
        System.out.println("Refund e-wallet
berhasil? " + checkout.refund(wallet,
ebook.getFinalPrice()));
        System.out.println("Saldo setelah refund:
Rp" + wallet.getBalance());
```

Kode Program 2.18

Sebuah Class ShopApp mendemonstrasikan seluruh proyek dalam main. Dua PhysicalProduct (kopi dan rendang) dan satu DigitalProduct (e-book) disimpan dalam List<Product>, dan getInfo() mencetak kategori lewat polimorfisme. CheckoutService lalu menghitung subtotal 250.000 dan berat 700 gram (e-book tidak dihitung karena bukan Shippable).

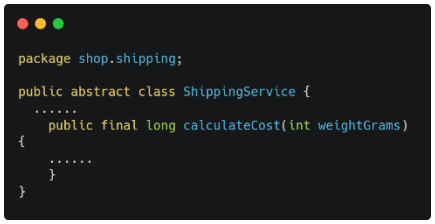
Total Reguler adalah 260.000 dan total Ekspres 268.000, karena kedua kurir bertipe ShippingService.

Pembayaran memakai tiga metode yang diterima checkout() tanpa perubahan kode karena parameternya bertipe PaymentMethod (*program to an interface*): e-wallet (saldo 300.000 menjadi 40.000), transfer bank, dan kartu kredit dengan limit 1.000.000. Cicilan kartu 3 bulan adalah 89.334 (dibulatkan ke atas). Refund e-book sebesar 45.000 dikembalikan ke e-wallet lewat checkout.refund(), sehingga saldo menjadi 85.000. Method itu hanya menerima RefundablePayment, jadi objek transfer bank ditolak compiler.

1.2.4 Latihan (L4)

Pada latihan 4 membahas final method dan final class dengan membuat beberapa class baru yang berhubungan dengan proyek java sebagai berikut.

a. Kunci rumus ongkir



```
package shop.shipping;

public abstract class ShippingService {
  ......
    public final long calculateCost(int weightGrams)
{
    ......
    }
}
```

Kode Program 2.19

Potongan kode ini adalah perubahan pada langkah L4.a, yaitu method calculateCost() pada ShippingService diberi final. Jika ada class yang mencoba meng-override calculateCost(), compiler menolaknya dengan pesan seperti cannot override ... overridden method is final, dan itulah error yang dilampirkan pada laporan L4.

b. Address immutable

```java
package shop.model;

public final class Address {
    private final String street;
    private final String city;
    private final String postalCode;

    public Address(String street, String city,
String postalCode) {
        if (street == null || street.isBlank()) {
            throw new
IllegalArgumentException("Street tidak boleh
kosong");
        }
        if (city == null || city.isBlank()) {
            throw new IllegalArgumentException("City
tidak boleh kosong");
        }
        if (postalCode == null ||
postalCode.isBlank()) {
            throw new
IllegalArgumentException("Postal Code tidak boleh
kosong");
        }

        this.street = street;
        this.city = city;
        this.postalCode = postalCode;
    }

    public String getStreet() {
        return street;
    }

    public String getCity() {
        return city;
    }

    public String getPostalCode() {
        return postalCode;
    }

    @Override
    public String toString() {
        return street + ", " + city + " " +
postalCode;
    }

}
```

Kode Program 2.20

Class Address adalah *final class* (L4.b), sehingga tidak dapat diturunkan. Tujuannya mencegah subclass seperti EditableAddress yang menambahkan setter dan merusak sifat immutable-nya. Ketiga field (street, city, postalCode) bersifat private final dan hanya punya getter tanpa setter, sehingga alamat tidak bisa berubah setelah dibuat. Karena itu alamat pesanan aman dibagikan ke beberapa objek Order tanpa ikut berubah. Konstruktornya menolak nilai null atau kosong pada ketiga field dengan IllegalArgumentException, dan toString() menggabungkannya menjadi teks seperti Jl. Moh Hatta No. 10, Padang 25121 yang dipakai pada struk.

1.2.5 Latihan (L5)

Pada latihan 5 membahas Nested class: static nested, inner, dan anonymous class dengan membuat beberapa class baru yang berhubungan dengan proyek java sebagai berikut.

a. Class Order (sebagian diberikan)

Berikut merupakan class order yang sudah terdapat di dalam modul praktikum pekan 6.

```java
package shop.model;

import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

public class Order {
    private final String orderId;
    private final Address shippingAddress;
    private final List<Line> lines = new ArrayList<>
();

    public Order(String orderId, Address
shippingAddress) {
        if (orderId == null || orderId.isBlank() ||
shippingAddress == null) {
            throw new
IllegalArgumentException("Order id and address are
required");
        }
        this.orderId = orderId;
        this.shippingAddress = shippingAddress;
    }

    public String getOrderId() { return orderId; }
    public Address getShippingAddress() { return
shippingAddress; }

    public void addLine(Product product, int
quantity) {
        lines.add(new Line(product, quantity));
    }

    public List<Line> getLines() { return
Collections.unmodifiableList(lines); }

    // TODO getItemsTotal(), createReceipt(), class
Line, class Receipt
}
```

Kode Program 2.21

Class Order mewakili satu pesanan dan merupakan kerangka awal L5.. Field orderId dan shippingAddress bersifat private final, dan konstruktornya menolak ID kosong atau alamat null. Alamat aman dipakai bersama karena Address adalah *final class* yang immutable. Daftar lines berisi item pesanan bertipe Line, yaitu *nested class* yang ditulis di L5.b, dan addLine() menambahkan item dengan membuat objek Line baru. Method getLines() mengembalikan Collections.unmodifiableList(lines), bukan list.

b. Lanjutan Clas Order

```java
    public static class Line {
        private final Product product;
        private final int quantity;

        public Line(Product product, int quantity) {
            if (product == null || quantity <= 0) {
                throw new
IllegalArgumentException("Product tidak boleh null
dan quantity harus lebih dari 0");
            }
            this.product = product;
            this.quantity = quantity;
        }

        public Product getProduct() {
            return product;
        }

        public int getQuantity() {
            return quantity;
        }

        public long subtotal() {
            return product.getFinalPrice() *
quantity;
        }
    }

    public long getItemsTotal() {
        long total = 0;
        for (Line line : lines) {
            total += line.subtotal();
        }
        return total;
    }
```

Kode Program 2.22

```java
    public class Receipt {
        public String toText() {
            StringBuilder sb = new StringBuilder();
            sb.append("STRUK PESANAN
").append(Order.this.orderId).append("\n");

            for (Line line : lines) {
                sb.append("-
").append(line.getProduct().getName())
                  .append("
x").append(line.getQuantity())
                  .append(" =
Rp").append(line.subtotal()).append("\n");
            }

            sb.append("Total pesanan:
Rp").append(getItemsTotal()).append("\n");
            sb.append("Kirim ke:
").append(Order.this.shippingAddress.toString());

            return sb.toString();
        }
    }

    public Receipt createReceipt() {
        return this.new Receipt();
    }
}
```

Bagian ini melengkapi Order pada L5.b dengan dua jenis nested class. Line adalah *static nested class* karena hanya menyimpan data item (product dan quantity) dan tidak membaca data Order, sehingga bisa dibuat tanpa objek Order. Konstruktornya menolak product null dan quantity yang tidak positif. Method subtotal() mengalikan getFinalPrice() dengan quantity, jadi diskon produk otomatis ikut terhitung lewat polimorfisme. Method getItemsTotal() menjumlahkan subtotal() semua Line.

Receipt adalah *inner class* (tanpa static) karena harus membaca field milik objek luarnya, yaitu orderId, shippingAddress, dan lines. Field itu diakses lewat Order.this, dan objek Receipt hanya bisa dibuat bila ada objek Order. Method toText() menyusun struk dengan StringBuilder: judul berisi ID pesanan, satu baris per item, total pesanan, dan alamat tujuan. Method createReceipt() mengembalikan this.new Receipt(), yaitu membuat inner object yang terikat pada pesanan ini. Di ShopApp, hal yang sama ditulis order.new Receipt().

c. Lengkapi ShopApp (Lanjutan dan terakhir)

```
Address address = new Address("Jl. Moh Hatta No.
10", "Padang", "25121");
Order order = new Order("ORD-0001", address);
order.addLine(coffee, 2);
order.addLine(ebook, 1);
Order.Receipt receipt = order.new Receipt(); //
inner object needs an outer object
System.out.println(receipt.toText());

PaymentMethod voucher = new PaymentMethod() { //
anonymous class
    private long remaining = 50_000;

    @Override
    public String getName() { return "Voucher Promo
Hari Ini"; }

    @Override
    public boolean pay(long amount) {
        if (amount <= 0 || amount > remaining) {
            return false;
        }
        remaining -= amount;
        return true;
    }
};
System.out.println("\n" + voucher.getName() + "
untuk e-book berhasil? "
        + checkout.checkout(List.of(ebook), regular,
voucher));
```

Kode Program 2.23

Potongan L5.c ini mendemonstrasikan dua jenis nested class di ShopApp. Pertama dibuat Address dan Order dengan dua item, kopi 2 buah (2 × 85.000 = 170.000) dan e-book 1 buah (45.000 setelah diskon), sehingga total pesanan 215.000. Struk dibuat dengan order.new Receipt(). Sintaks ini diperlukan karena Receipt adalah *inner class*, jadi objeknya hanya bisa dibuat dengan objek Order sebagai pengaitnya (*inner object*

needs an outer object). Berbeda dengan Line, *static nested class* yang bisa dibuat tanpa Order. Hasil toText() memuat ID pesanan, rincian item, total, dan alamat Jl. Moh Hatta No. 10, Padang 25121.

Voucher dibuat sebagai *anonymous class*, yaitu implementasi PaymentMethod sekali pakai tanpa nama class dan tanpa file terpisah. Field remaining (saldo 50.000) hanya dimiliki objek ini, dan pay() gagal jika jumlah melebihi sisa. Karena checkout() hanya mengenal tipe PaymentMethod, voucher dapat langsung dipakai (*program to an interface*). Pembayaran e-book berhasil (true) karena e-book tidak punya berat sehingga ongkirnya 0, dan totalnya 45.000 tidak melebihi saldo voucher. Sisa saldo voucher menjadi 5.000.

2. Uji Pemahaman

Pada bagian ini akan membahas tentang pesan error yang diminta pada modul untuk pada L1 (membuat objek Product), L3 (refund lewat transfer bank), dan L4 (override calculateCost).

2.1 Latihan 1

Pada latihan 1 diminta untuk menulis p = new Product("X", "X", 1000); disebuah main.

```
public static void main(String[] args) {
        Product p = new Product("X", "X", 1000);
```

Muncul pesan eror Ketika mengetikkan kode tersebut sebagai berikut.

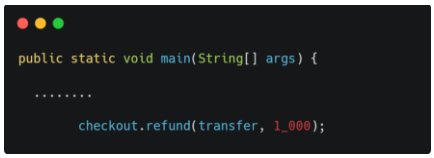
```
ShopApp.java src\shop\app 3
Cannot instantiate the type Product Java(16777373) [Ln 21, Col 25]
Product is abstract; cannot be instantiated (errors(1): 21:25-21:32) [Ln 21, Col 25]
Variable p is never read (hints(1): 21:17-21:18) [Ln 21, Col 17]
```

Artinya compiler menolak new Product(...) karena Product adalah abstract class. Class abstract memiliki method tanpa isi (category()), sehingga objeknya

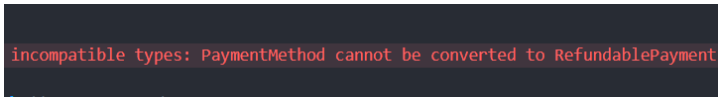
belum lengkap dan tidak boleh dibuat. Objek hanya bisa dibuat dari subclass konkret seperti PhysicalProduct, yang sudah mengisi semua method abstract.

2.2 Latihan 3

Pada latihan 3 diminta untuk coba checkout.refund(new BankTransferPayment("Bank Nagari"), 1_000);. Catat error compiler-nya, lalu jadikan komentar.



```
public static void main(String[] args) {
  ........
        checkout.refund(transfer, 1_000);
```



```
incompatible types: PaymentMethod cannot be converted to RefundablePayment
```

Artinya refund() hanya menerima tipe RefundablePayment, tetapi yang dikirim bertipe PaymentMethod. Variabel transfer dideklarasikan sebagai PaymentMethod, dan BankTransferPayment memang tidak mengimplementasikan RefundablePayment, sehingga tidak ada konversi yang sah. Bagian int pada pesan tidak bermasalah, karena 1_000 otomatis diperlebar menjadi long. Yang menolak adalah tipe pertamanya. Manfaatnya, kesalahan “refund lewat transfer bank” tertangkap saat kompilasi, bukan saat program berjalan.

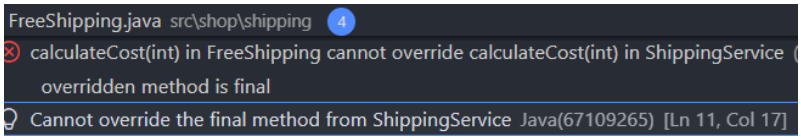
2.3 Latihan 4

Pada latihan 4 diminta untuk Uji: buat untuk sementara class FreeShipping extends ShippingService yang meng-override calculateCost.



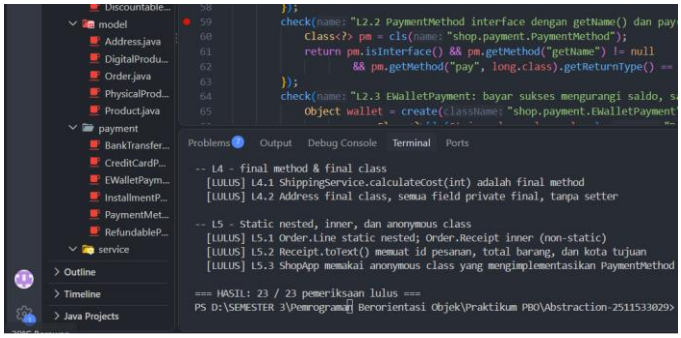
```
package shop.shipping;

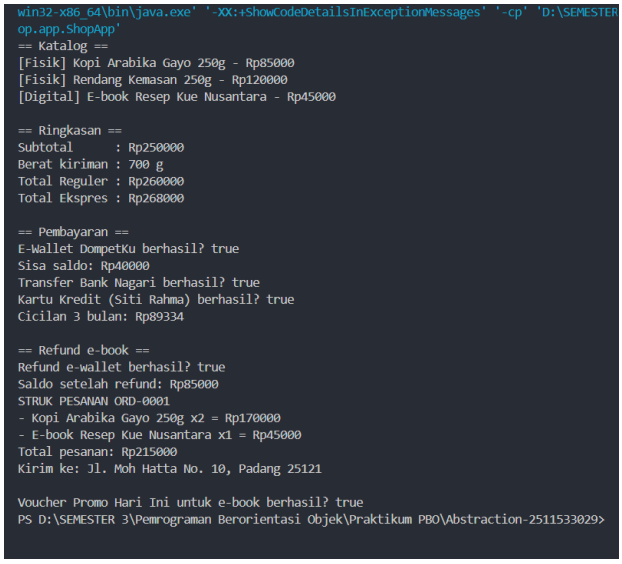
public class FreeShipping extends ShippingService {
  ......
    @Override
    public long calculateCost(int weightGrams) {
        return 0;
    }
}
```



Cannot override the final method from ShippingService, Artinya method calculateCost() di ShippingService ditandai final, sehingga subclass dilarang menimpanya.

Berikut hasil selfcheck dan output shopAPP.





3. Link Repository

Link repository untuk semua kode program pada pekan 6 sebagai berikut.

sofianarbai/PrakPBO_C_2511533029 at after-uts